Page Matrix — Technical Reference

Engineering-oriented reference for the **dashboard**: how it scans pages, how it scores them, how it rolls them up, the site checks, and the Phase 5 UI. Since Phase 3 the dashboard uses **the same check engine as the panel** — the old 5-pillar model (`site-score.ts`: Crawlability, Structured, Answerability, Authority, Retrievability) is gone. The checks themselves (IDs, thresholds, credit) are documented once, in `PAGE-CHECKS-TECHNICAL.md`.

Source of truth:

- `src/hooks/useSitePages.ts` — page enumeration, scanning, the score cache, site-check inputs
- `src/lib/checks/` — the engine (see `PAGE-CHECKS-TECHNICAL.md`)
- `src/lib/checks/view-model.ts` — `PageScoreSummary`, `CATEGORY_META`, Fix first grouping, status words
- `src/lib/site-aggregate.ts` — site roll-up
- `src/app/dashboard-widget-extension/page.tsx` and `src/components/dashboard/*.tsx` — the UI (one component per tab)
- `src/hooks/useScoreHistory.ts` — the last 8 site scores
- `src/hooks/useSiteType.ts` — the Site type setting
- `src/types/dashboard.ts` — `PageMatrixRow`, `MatrixFilter`

Scoring model

Each page is scored by `runPageChecks` with a context from `buildPageContext` — the same function the panel uses — and summarised by `summarisePageReport` into a `PageScoreSummary`:

Field	Meaning
`score`	0–100, or `null` when the gate blocks the page or nothing could be scored
`label`	`not-ready` / `almost-there` / `agent-ready` from `labelForPct` (80 / 60), or `null`
`blocked`, `gateReason`	the gate (P1): Sitecore Robots Meta Tag field, or a rendered noindex / nosnippet / max-snippet:0
`categories`	earned and max points for `reach`, `read`, `understand`, `trust` (max 0 when nothing in that category was scored)
`issues`	every scored check that isn't passing: `{ id, status, gain }` with `status` "warning" or "fail" and `gain` = `maxPoints − points`. Empty when blocked. Feeds Fix first

Pools: Essential checks share 80 points, Recommended 20, Bonus adds up to 5, capped at 100. N/A and not-checked results are left out of the pools. Full detail in `PAGE-CHECKS-TECHNICAL.md`.

Rating bands — `labelForPct`, `src/lib/score-thresholds.ts`

Label	Range	Shown as (`LABEL_TEXT`)
`agent-ready`	≥ 80	Agent-ready
`almost-there`	60–79	Almost there
`not-ready`	< 60	Not ready

The same `READY_PCT` (80) and `ALMOST_PCT` (60) drive the score rings, the heatmap cells, the verdicts and the site roll-up.

Scanning a page (`scanOne`, `useSitePages.ts`)

- **Context first:** `application.context` is read before any XM Cloud call, and its Context ID (`resourceAccess[].context.preview`, else `.live`) is sent as `sitecoreContextId` on `xmc.sites.retrieveSite` and every authoring GraphQL call. The console line `[sitePages] application.context →` shows the Marketplace app type (`custom` / `public`), the extension points (for dashboard blocks, each widget's route is under `blocks`, and the top-level route is null), and whether a Context ID was found. It names fields one by one, because the raw context can hold the editing secret. `[sitePages] host.state →` shows which Sitecore host runs the widget: `isNull: true` for the Cloud Portal, `environment` and `language` for SitecoreAI.

- **Host refusing authorization:** if the Sitecore host refuses a call with "getAccessToken method not provided" (plan section 13, item 1), `isHostAuthError` (`src/lib/xmc-errors.ts`) recognises it. The Overview then says Sitecore didn't authorize the request, instead of falling back to the content-tree walk and reporting a missing Home item. Other errors, including a plain 401 from an API, keep the existing fallbacks.
- **Enumeration:** authoring GraphQL walks the site's Home item 5 levels deep; items with `__Renderings` or `__Final Renderings` are pages. For each it reads `template { name }`, `__Updated`, and the fields the checks read, by the shared names in `SCORING_FIELD_NAMES` (`pageTitle`, `MetaDescription` — overridable with `NEXT_PUBLIC_FIELD_TITLE` / `NEXT_PUBLIC_FIELD_META_DESCRIPTION` — and `robotsMetaTag`). `pageFieldsFromValues` normalises them into `row.fields`, exactly as the panel does. The OG description is also read, only as the llms.txt summary fallback.
- **Scan:** `/api/preview-fetch` for the live HTML, then `/api/ai-check` with `ParsedHtml.mainText` (on `ANTHROPIC_MODEL_BULK`, cached by content hash in localStorage, shared with the panel, and on the server). PageSpeed doesn't run in the dashboard, so P7 is "not checked" and left out of the score. The dashboard is **live-only**: it has no Page Builder draft (`getPageHTML` works only in Page Builder), so it passes no `draft` to `buildPageContext`, and P3 is "not checked" ("Page Builder only") and left out too. The same inputs give the panel's score; the panel's can differ when it scores a draft (PAGE-CHECKS-TECHNICAL, "Draft and live"). `mainText` now includes collapsed FAQ answers, so accordion pages get a new AI cache key once.
- **Concurrency:** 5 pages at a time; all unscanned rows scan in the background after enumeration.

Score cache

Summaries are cached in localStorage under `aiDiscovery:scores:v5:<siteId>`, keyed by page URL and reused while the page's `__Updated` is unchanged. **v5** (Phase 5) adds `issues` and the new label names; **v4** was the unified engine. A new version makes the dashboard rescan every page once. The AI results have their own content-hash cache, so pages whose text hasn't changed don't call `ai-check` again. **Scan site** clears the cache and re-fetches the site-check inputs.

Site roll-up (`site-aggregate.ts`)

- **Overall score:** mean of the page scores. Unscanned and **blocked** pages are left out; blocked pages are counted separately ("N Blocked" in the summary).
- **Category cards:** for each category, the mean of `earned ÷ max` over pages where that category was scored. A page "needs work" in a category below 60% (`FAIL_THRESHOLD`, the Almost there line; it was 40% before Phase 5). The "What's included" list comes from the check registry (`pageCheckTitles`), so it always matches what's scored; P3 and P7 are marked "(Page Builder panel only)".

- **Pages heatmap:** a category cell shows `round(earned ÷ max × 100)`, coloured by `labelForPct`, or N/A when nothing in it was scored. Blocked pages show the gate reason across the cells and sort with the unscanned ones.

Fix first (`view-model.ts`)

- **Page issues:** `groupPageIssues` groups every page's `issues` by check ID. `gainEach` is the mean gain, rounded ("+N pts each"); groups under 1 point are dropped. The worst status wins (any `fail` → Fix). Sorted by the total gain across pages.
- **Site issues:** `siteIssueGroups` turns the site checks that need work into groups that sort first ("Affects every page").
- **Buttons:** site issues and Developer page issues get **Send to developer**, which copies `issueNote` (the issue, why, and each page's URL; for a site issue, `developerNote` with its fix and snippet). Author page issues get **Review pages**, which opens the Pages tab filtered to `pageIds`. Bulk "Generate for all" is Phase 9.
- The hero sentence comes from the same groups: the number of site issues, or the biggest page issue, and how many issues authors can fix.

History (`useScoreHistory.ts`)

`useSiteHistory` keeps the last 8 site scores in localStorage (`aiDiscovery:siteHistory:v1:<siteId>`). It adds one when a scan finishes in this session (pages were scanning, now none are), not on every load from cache. The hero shows "▲ N since last scan" (the last two entries) and the trend line from two scans on. Phase 8 decides on shared storage for "What changed".

Agent access tab: site checks

Once per site, the hook fetches `/api/robots-check` and the home page (`/api/preview-fetch` on `<domain>/`), not awaited by enumeration. The page then runs `runSiteChecks` with:

- the home page HTML (null when it couldn't be fetched → S7 is "Couldn't check", N/A),
- robots.txt,
- the organisation for the snippet (`getOrganizationInfo`, `NEXT_PUBLIC_ORG_*` with the site's name and domain as fallbacks) and the configured organisation as the expected value (`getConfiguredOrganization`),
- the site type.

Site checks never feed page scores. Phase 3 has **S2** `reach.robots-ai-rules` (the old 6-crawler robots.txt signal, moved as is) and **S7** `understand.org-schema` (home page Organization JSON-LD, graded with the four agreed rules). Phase 6 adds the rest.

`AgentAccessTab.tsx` lists them as a checklist (status word, title, tier, message, and a button when they need work: S2 → **Open robots.txt help** in Files, S7 → **Copy snippet**). A third row, **About, contact and policy pages are linked**, shows `/api/footer-check` (fetched when the tab first opens; 5 or more of the 6 signals is Pass, 3 or more Improve). It's labelled "Not scored yet". The table beside it shows the six crawlers' robots.txt result from `siteInputs.robotsTxt` ("unknown", not mentioned, is allowed by default). Live requests per user agent are Phase 6.

Files tab (`FilesTab.tsx`): the llms.txt generator (`buildLlmsTxt` , now with `whenToUse` → `## When to use this site` , and a checkbox for the fixed 60 threshold), and robots.txt help: the `User-agent: … / Allow: /` lines for each blocked crawler. The Content-Signal and bot-group generator is Phase 6.

Site type (Content site / Commerce / App or API) is a select in the header, stored per site in localStorage (`aiDiscovery:siteType:v1:<siteId>`). It decides which Bonus protocol checks apply (S11 App/API, S12 Commerce), which arrive in Phase 6, so it has no visible effect yet.

What changed from the pillar model

Old pillar signal	Now
Crawlability: AI crawler permissions	S2 `reach.robots-ai-rules` (site check)
Crawlability: canonical	P2 `reach.canonical` (Essential)
Crawlability: robots meta	P1 gate — blocks the score instead of costing 4 points
Structured: FAQPage / Article / BreadcrumbList	P10 `understand.json-ld` — now validates required properties, reads `@graph` and `@type` arrays, expects FAQPage from real Q&A pairs instead of question H2s
Structured: Organization (home page)	S7 `understand.org-schema` (site check)
Answerability: headings	P8 `understand.headings`
Answerability: question H2s	P13 `understand.question-headings`
Answerability: reading grade + paragraph length	P14 `understand.readability`
Authority: AI uniqueness	P16 `trust.uniqueness`
Authority: content depth	P19 `trust.depth`
Authority: freshness	P18 `trust.freshness` (adds JSON-LD `dateModified`; reads Sitecore's compact date format)
Authority: descriptive anchors	P20 `trust.anchor-text` (prefix rule narrowed, SPIKE-FINDINGS §2)
Retrievability: title + description	P9 `understand.title-meta` (Sitecore field first; title band now 50–60)
Retrievability: OG tags	P11 `understand.metadata` (adds `html lang`, `og:type`)
Retrievability: landmarks	P4 `read.landmarks`
Retrievability: image alt	P15 `understand.image-alt` (N/A without images)

New in the dashboard because it now shares the panel's checks: P11's lang and og:type, P12 FAQ answer quality, P17 author (article templates), P21 follow-up gaps (Bonus).

UI (Phase 5)

Blok tokens and components (`src/components/ui`), `@mdi/js` icons, and the shared components in `src/components/shared` (also used by the panel; see PAGE-CHECKS-TECHNICAL). Tabs are Radix tabs: arrow keys move between them. A URL hash (`#pages` , `#agent-access` , `#citations` , `#files`) opens that tab; it's read after mount so hydration matches. Header: site name and domain, Site type, "Last scan N ago" (the newest `lastScannedAt`), and **Scan site** (`clearAndRefresh`). The page-list error shows on Overview and Pages with **Retry**; the other tabs only need the domain.

Demo mode (`?demo=1`)

`makeTestSitePages()` (`src/lib/rule-engine/test-fixture.ts`) builds 12 rows by running the **real engine** on variants of the panel's demo page, so the numbers are consistent: 3 Not ready, 3 Almost there, 5 Agent-ready and 1 blocked (noindex), a site score of 74. The row "Blog: content strategy" is the panel's demo page unchanged, scored on live HTML only (89). The panel demo also scores its demo draft (`makeTestDraft` : an edited heading and client-rendered Related Articles cards), so it shows 81, with the "Draft differs" banner and a P3 warning. `makeTestSiteContext()` gives the Agent access tab a BrandX-style home page with no JSON-LD (S7: 0, warning, snippet) and a robots.txt that blocks GPTBot (S2: half credit).

Demo mode never starts the Marketplace client (`useMarketplaceClient` checks `isDemoUrl()`), in the dashboard or the panel. So it makes no SitecoreAI calls and can't write to a real page, even if `?demo=1` is opened inside a Sitecore extension point. The panel doesn't auto-scan in demo mode either: the fixture already holds the HTML, the draft and the AI results, so nothing fetches the fixture URL, reads `getPageHTML` or calls `ai-check` on load. Agent access and Files use the fixture's robots.txt and `makeTestFooterTrust()` , so the dashboard fetches nothing in demo mode. The trend line uses `DEMO_SITE_HISTORY_BEFORE` plus the current score, and **Scan site** is off. Generate meta and Citation forecast still call Claude when clicked.

Key files

Aspect	File
Enumeration, scanning, score cache, site-check inputs	`src/hooks/useSitePages.ts`
Host authorization refusal	`src/lib/xmc-errors.ts`
Demo mode check (`?demo=1`)	`src/hooks/useDemoMode.ts`
Check engine, registry, scoring	`src/lib/checks/`
Summary shape, category labels, Fix first grouping, developer notes	`src/lib/checks/view-model.ts`
Site roll-up	`src/lib/site-aggregate.ts`
Dashboard UI	`src/app/dashboard-widget-extension/page.tsx` , `src/components/dashboard/{OverviewTab,PagesTab,AgentAccessTab,FilesTab,Sparkline}.tsx`
Shared UI (rings, chips, status icons, buttons)	`src/components/shared/` , Blok in `src/components/ui/`
Score history	`src/hooks/useScoreHistory.ts`
llms.txt builder	`src/lib/llms-txt/build.ts`

Aspect	File
Site type setting	`src/hooks/useSiteType.ts`
Dashboard types	`src/types/dashboard.ts`
Demo data	`src/lib/rule-engine/test-fixture.ts`